

图形绘制技术

实验报告

基于 MoerEngine 的二次元非真实感渲染与
风格化后处理

学生姓名

叶同学

学号

学院

计算机学院

完成日期

2026 年 7 月 15 日

目录

1	实验目的	1
2	实验内容与要求	1
3	实验环境	2
3.1	硬件环境	2
3.2	软件环境	2
3.3	实验场景与资源	3
4	实验原理与总体方案	3
4.1	光栅化渲染流程	3
4.2	系统总体设计	4
4.3	图形输出方案	5
5	关键算法与实现	6
5.1	二分色阶 Toon 着色	6
5.2	基于深度与法线的描边	6
5.3	像素化	7
5.4	半调	7
5.5	故障艺术	8
5.6	素描	8
5.7	水彩	8
5.8	内部纹理 PNG 导出	9
6	实验过程	9
6.1	场景准备	9
6.2	参数设置	10
6.3	实验步骤	10
7	实验结果与分析	10
7.1	PBR、Toon 与描边对比	10
7.2	五种单项后处理结果	11
8	问题分析与解决方法	13
8.1	Toon 输出接近黑色	13
8.2	最小化后只显示天空	13
9	实验局限与改进方向	13

10 实验结论	13
11 源码仓库	14
12 课程总结与建议	14
资源来源与许可	14
参考文献	15

1 实验目的

本实验以 MoerEngine 实时渲染引擎为基础，在 Vulkan 光栅化后端中完成一套面向二次元角色的非真实感渲染（Non-Photorealistic Rendering, NPR）流程。实验首先验证 Mizuki 角色在引擎中的导入、材质解析和场景渲染，然后在延迟光照阶段实现二分色阶 Toon 着色，并利用深度与法线缓冲实现角色和场景的屏幕空间描边。在此基础上，实验进一步实现像素化、半调、故障艺术、素描和水彩五种风格化后处理，使每种效果均具有独立配置、独立渲染管线和独立执行开关。

为稳定采集实验结果，本实验扩展了 MoerEngine 的图形输出链路。程序从风格化后的最终场景纹理执行图形处理器（Graphics Processing Unit, GPU）回读、线性颜色到 sRGB 的转换和 PNG 编码；窗口最小化期间继续执行完整光栅渲染，并跳过交换链呈现（Present）。

实验涉及延迟渲染中的颜色、法线与深度资源，Toon 明暗量化、屏幕空间轮廓检测、五种全屏后处理，以及交换链、内部纹理、GPU 回读、颜色空间和图像编码。

注：实验完整实现已经公开于 <https://github.com/FeNgkAIY/MoerEngine-NPR>，包括二分色阶 Toon 着色、基于深度与法线的描边、像素化、半调、故障艺术、素描、水彩和内部纹理 PNG 导出的实现。对应版本为 npr-final-project 分支的 5de587c5 提交。

2 实验内容与要求

课程作业要求导入并正确渲染二次元角色，实现二分色阶 Toon 着色，实现角色与场景描边，并从像素风格、半调或仿色图案、故障艺术、素描和水彩五类后处理中至少完成一种。本实验在必做内容之外完成了全部五种后处理，每种效果均作为独立模块启用。表 2.1 给出要求、实现方法和验证方式之间的对应关系。

表 2.1 实验要求与完成情况

实验内容	实现方法	验证方式
二次元角色导入	导入 Mizuki glTF 角色，并与 Sponza 组成实验场景	角色正面渲染图
二分色阶 Toon 着色	使用主方向光、阴影项、明暗阈值与暗部比例生成两级明暗	PBR 与 Toon 对比
角色与场景描边	比较八邻域的线性深度差和法线夹角差	Toon 与描边对比
风格化后处理	独立实现像素化、半调、故障艺术、素描和水彩	同场景结果总览
图形输出改造	最终场景纹理回读、sRGB 转换和 PNG 编码	最小化导出文件

实验保留物理渲染（Physically Based Rendering，PBR）结果作为对照，并实现 Toon 着色、角色与场景描边以及像素化、半调、故障艺术、素描和水彩五种后处理。

3 实验环境

3.1 硬件环境

实验在配备 AMD Ryzen 7 7800X3D 处理器、AMD Radeon RX 9070 图形处理器和 32 GB 内存的计算机上完成，硬件配置见表 3.1。Vulkan 光栅化渲染使用 AMD Radeon RX 9070。

表 3.1 硬件环境

项目	配置
处理器	AMD Ryzen 7 7800X3D 8-Core Processor
图形处理器	AMD Radeon RX 9070
内存	32 GB

3.2 软件环境

MoerEngine 是一个实时渲染引擎。本实验使用其 Vulkan 光栅化路径，在延迟光照和全屏后处理阶段实现 Toon 着色、描边与五种风格化效果 [1]。

表 3.2 软件环境

项目	版本或配置
操作系统	Windows 10 Pro 10.0.19045
渲染引擎	MoerEngine, 实验基线提交 e89d299f
图形后端	Vulkan, 配置接口版本 1.3
着色器语言	高级着色语言 (High-Level Shader Language, HLSL)
着色器编译	DirectX Shader Compiler (DXC) 输出 SPIR-V
构建类型	Debug
输出分辨率	1920×1080

3.3 实验场景与资源

Mizuki是本实验从 MoerEngine 随附测试资源中选用的二次元角色。该模型的面部、帽子、服装和配饰具有明确色块，适合观察 Toon 明暗分区、人物轮廓和不同后处理对局部辨识度的影响。模型原始来源为 Sketchfab 的 “With This Pitch...! Mizuki”，作者为雨宫レン [2]。

Sponza 用于构造包含拱门、帷幕、石墙、植物和吊灯的复杂背景。复杂几何和多尺度纹理使描边、素描排线、半调网点和水彩保边平滑更容易观察。该场景取自 MoerEngine 本地资源中的 Sponza 场景。

组合场景在启动前将预变换后的 Mizuki 图元加入 Sponza 网格。角色位于中央通道并面向相机，角色世界高度约为 2.17 m。

4 实验原理与总体方案

4.1 光栅化渲染流程

MoerEngine 光栅化路径首先在几何阶段生成几何缓冲区 (Geometry Buffer, G-buffer)，其中包含基础色、法线、金属度/粗糙度/环境光遮蔽参数和深度。延迟光照阶段根据 G-buffer 和场景光源生成高动态范围 (High Dynamic Range, HDR) 颜色，随后经过抗锯齿、Bloom 和色调映射。PBR 用作对照；正式风格化结果使用 Toon 着色，并在色调映射后执行描边和一种后处理。

图 4.1 给出实验采用的数据流。最终风格化纹理有两个去向：一条路径进入编辑器界面合成并通过交换链显示；另一条路径在界面合成之前执行回读与 PNG 编码。

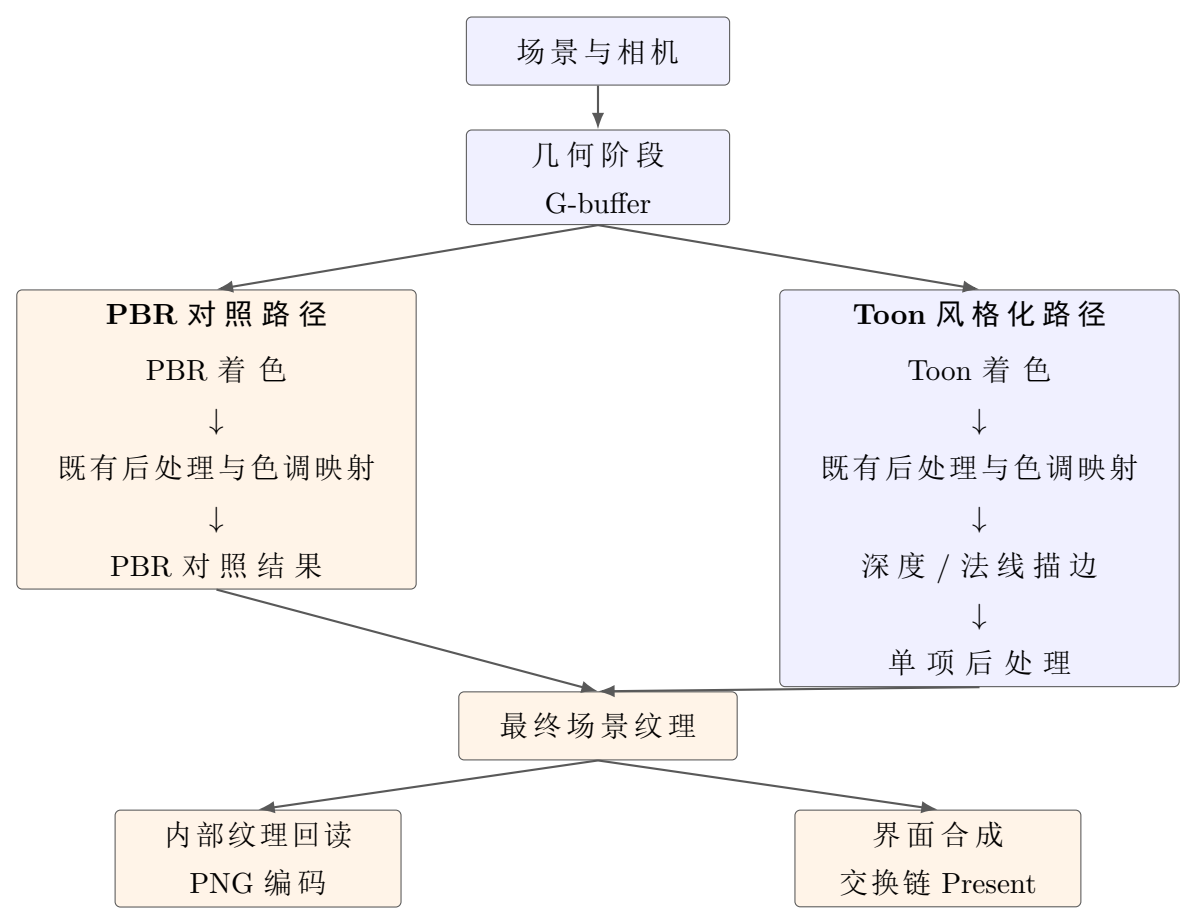


图 4.1 PBR 对照路径与 Toon 风格化路径

4.2 系统总体设计

正式结果包含三个处理阶段。Toon着色将连续光照量化为明暗两级；描边根据深度和法线差生成角色与场景轮廓；单项后处理再执行像素化、半调、故障艺术、素描或水彩。处理阶段使用 stylized_work 和 stylized_output 两张纹理交替写入，避免单个阶段同时读写同一纹理。

图 4.2 展示 Toon、描边和单项后处理之间的数据关系。若没有选择扩展后处理，Toon 与描边结果直接写入统一的 stylized_output；若风格化模块均关闭，则复制模块负责输出原颜色。这样可以保证后续显示与导出始终读取稳定的最终纹理。

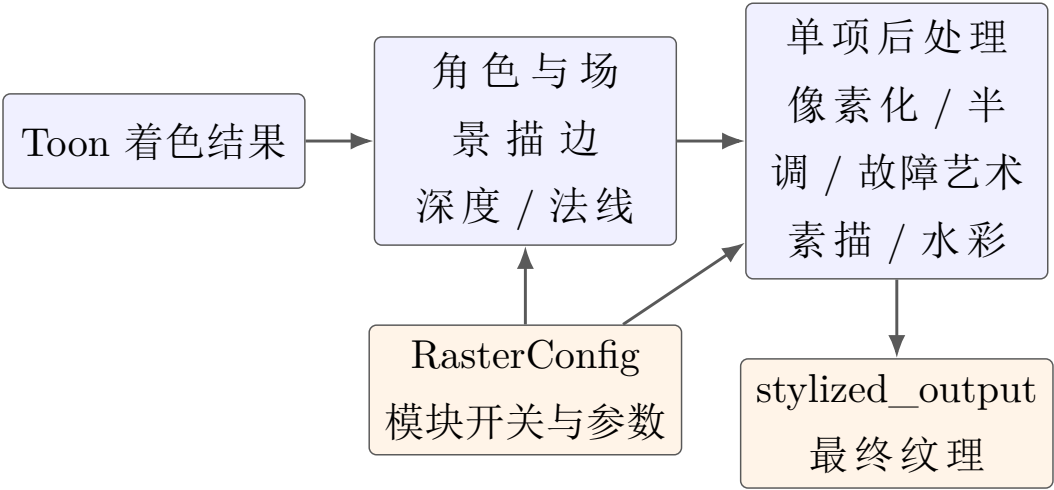


图 4.2 Toon、描边与单项后处理结构

4.3 图形输出方案

场景纹理经过编辑器界面合成后通过交换链 Present 显示。实验结果直接读取界面合成前的场景纹理，输出内容只包含渲染场景。

内部纹理导出分支如图 4.3 所示。导出源是完成色调映射和风格化后的内部最终场景纹理，获取位置早于编辑器界面合成。程序正常创建窗口和交换链；如果请求最小化导出，则在交换链创建后最小化主窗口。每帧仍执行场景更新、阴影、几何、光照、既有后处理和风格化处理，只将 skip_present 设为真，使帧末尾不调用 Present。该方式不是无窗口渲染，而是在保留窗口与交换链的条件下，将最小化状态与内部渲染解耦。

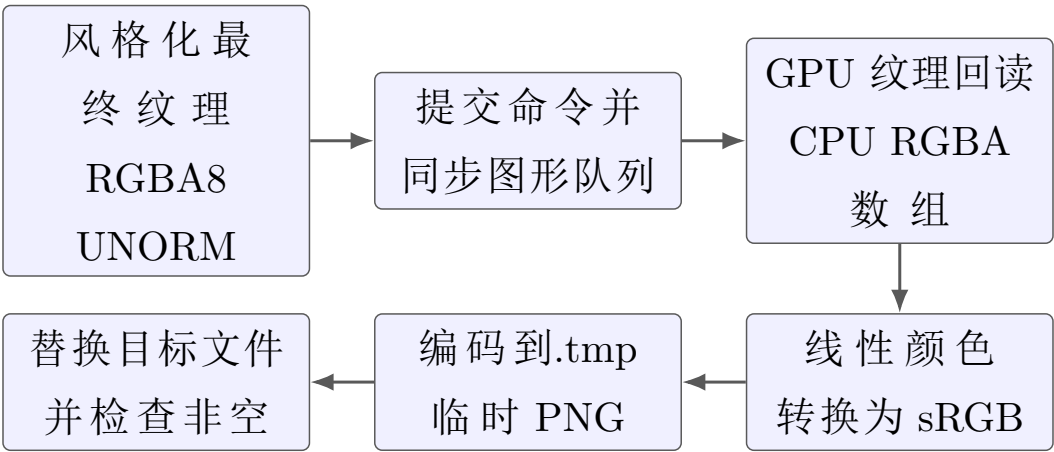


图 4.3 内部最终纹理 PNG 导出流程

导出期间固定 Scene Color 的内部尺寸为完整渲染分辨率，避免编辑器面板在窗口最小化后缩小，从而改变相机宽高比和视锥剔除结果。导出请求还支持预热帧数，用于等待自动曝光等时间相关状态收敛。本报告中的正式对比图均在 1,200 帧预热后导出。

5 关键算法与实现

5.1 二分色阶 Toon 着色

传统 PBR 通过法线分布函数、几何遮蔽和菲涅耳项连续描述光照反射，适合表达材质的物理质感。Toon 着色则主动压缩连续明暗，使形体通过少量稳定色阶表达。非真实感光照通过控制明暗和色彩组织提高形状的图示性，相关思想在技术插画和卡通渲染中已有广泛应用 [3-5]。

本实验在延迟光照阶段保留同一套 G-buffer，使用单位法线 $\mathbf{N}$ 、主方向光方向 $\mathbf{L}$ 和阴影项 s 计算光照量：

$$l = \text{saturnate}(\mathbf{N} \cdot (-\mathbf{L})) s. \quad (5.1)$$

光照量通过阈值 τ 量化为明部和暗部：

$$t = \begin{cases} 1, & l \geq \tau, \\ r_s, & l < \tau, \end{cases} \quad (5.2)$$

其中 r_s 为暗部比例。本实验默认 $\tau = 0.45$, $r_s = 0.45$ 。引擎处于 HDR 光照链路，若仅输出基础色乘色阶，结果会明显低于场景中方向光的辐射量级。因此实现遍历方向光，将其强度和颜色按 $1/\pi$ 缩放，得到 $\mathbf{E}_d$ ，最终颜色为

$$\mathbf{C}_{\text{toon}} = \mathbf{C}_{\text{albedo}} \mathbf{E}_d t. \quad (5.3)$$

该方法保持方向光颜色和强度的影响，同时将表面响应限制为两个明暗区间。

5.2 基于深度与法线的描边

屏幕空间描边从 G-buffer 读取 Reverse-Z 深度和打包法线。深度越接近相机，Reverse-Z 值越大；天空像素深度接近零。当前实现按相机近裁剪面 n 和远裁剪面 f 将原始深度 d 转为线性深度：

$$z(d) = \frac{nf}{f + (1 - d)(n - f)}. \quad (5.4)$$

对中心像素和邻域像素的线性深度 z_a, z_b ，使用对称相对差

$$D(a, b) = \frac{2|z_a - z_b|}{\max(z_a + z_b, 2n)} \quad (5.5)$$

降低绝对距离对阈值的影响。法线差定义为

$$N(a, b) = 1 - \text{saturnate}(\mathbf{n}_a \cdot \mathbf{n}_b). \quad (5.6)$$

着色器在水平、垂直和四个对角方向共采样八个邻居，分别取最大深度差和最大法线差，再通过平滑阈值生成深度边缘和法线边缘。二者取最大值后将原颜色插值到描边颜色。深度边缘能够检测物体轮廓和遮挡关系，法线边缘能够补充同一物体内部的折角。该方法与基于图像空间辅助缓冲区提高形状可读性的思路一致 [6]。

本实验默认描边宽度为 1 px，深度阈值为 0.02，法线阈值为 0.15，颜色为黑色。深度和法线来源可独立开关；天空像素直接返回原颜色，避免在无几何区域产生错误轮廓。

5.3 像素化

像素化模块将屏幕划分为边长为 k 的方形单元，每个像素统一读取所在单元中心的颜色。对整数像素坐标 $\mathbf{p}$ ，采样坐标为

$$\mathbf{p}' = \left\lfloor \frac{\mathbf{p}}{k} \right\rfloor k + \left\lfloor \frac{k}{2} \right\rfloor. \quad (5.7)$$

课程教程使用额外摄像机把场景写入低分辨率渲染纹理，关闭抗锯齿，将纹理过滤设置为点采样（Point），再把纹理显示到主摄像机；降低渲染纹理分辨率会增大屏幕上的像素颗粒 [7]。本实验需要保持主渲染和 PNG 导出的 1920×1080 分辨率，因此没有改变场景渲染尺寸，而是在全屏后处理中把采样坐标量化到像素块中心。两种方法都让一片屏幕像素共享同一颜色，本实验的方法可以直接通过像素块尺寸控制颗粒大小。实验使用 4 px 像素块。

5.4 半调

半调使用离散网点的数量、面积和颜色近似连续色调。数字半调和有序阈值方法通过空间占空比表达连续灰度，Bayer 阈值矩阵是其中常用的确定性排列 [8, 9]。

本实验先将像素坐标绕图像中心旋转指定角度，再按单元尺寸计算网格索引。每个单元在中心周围执行 3×3 采样，将线性颜色转到显示空间后计算局部平均颜色 $\mathbf{c}$ 。感知亮度为

$$Y = 0.2126c_r + 0.7152c_g + 0.0722c_b, \quad d = 1 - Y, \quad (5.8)$$

其中 d 是暗度。着色器通过平滑阈值和幂函数将暗度映射为目标墨量 c ，再使用

$$\rho = \sqrt{c} \quad (5.9)$$

同时控制有效网点密度和单点覆盖量。当前 4×4 Bayer 阈值矩阵包含 16 个均匀分布阈值；单元阈值低于 ρ 时生成网点。单点覆盖量上限为 0.74，低于内切圆面积 $\pi/4$ ，因此实际圆点半径按 $r = \sqrt{\rho/\pi}$ 计算。目标墨量超过单点上限时，着色器保持圆点尺寸上限并提高有效网点密度。

该限制避免暗部被单个大网点完全覆盖。网点之外输出暖色纸张底色，网点内部

输出由局部颜色调制的深色墨水；强度为 1 时画面由纸张和网点重构。实验参数为 5 px 单元、45° 旋转角度和 0.75 边缘柔化。

5.5 故障艺术

故障艺术模块按垂直坐标把画面划分为水平条带，并使用帧序号离散化时间。哈希噪声决定条带是否激活和位移方向；激活条带沿水平方向偏移，红、绿、蓝三个通道分别使用 $o + c$ 、 o 和 $o - c$ 的采样位置，从而产生色散。条带内部再叠加隔行扫描线衰减。默认参数为强度 0.8、速度 1.0、色差偏移 3 px、条带高度 24 px。实现参考了课程提供的故障风教程 [10]。

该方法保持单帧内条带结构稳定，但随离散时间步变化。画面中的建筑直线、帷幕边缘和人物轮廓会在不同条带中发生横向错位，RGB 通道分离进一步强化数字信号异常感。

5.6 素描

素描模块先计算显示空间感知灰度和暗度，再组合三类边缘：原始深度差、法线夹角差以及颜色亮度梯度。三类边缘按固定权重累加并乘边缘强度。为避免得到机械的连续描线，边缘还通过低频哈希噪声打断。

排线使用屏幕空间的正对角线、负对角线和水平方向。亮度越低，依次激活更多方向，形成单向、交叉和密集交叉排线。第三组方向取(1,0)。每条线沿切向分段，并用哈希值控制连续性。最终颜色在暖白纸张底色、灰度底稿和深色铅笔颜色之间插值，同时叠加轻微纸张颗粒。默认强度为 1.0，排线间距为 11 px，边缘强度为 1.5。

5.7 水彩

水彩模块的核心是桑原滤镜（Kuwahara Filter）[11]。以当前像素为中心，将半径 r 的邻域分为左上、右上、左下和右下四个重叠象限。对每个象限 q 计算颜色均值

$$\mu_q = \frac{1}{N_q} \sum_{i \in q} x_i \quad (5.10)$$

以及逐通道方差

$$\sigma_q^2 = \frac{1}{N_q} \sum_{i \in q} x_i^2 - \mu_q^2. \quad (5.11)$$

着色器选择 $\sigma_{q,r}^2 + \sigma_{q,g}^2 + \sigma_{q,b}^2$ 最小的象限均值作为输出。低方差区域通常位于同一颜色块内部，因此能够平滑纹理；跨越边缘的象限方差较大，不易被选择，从而保留主要轮廓。

选定颜色再执行轻度色阶量化，并乘以粗细两级噪声模拟纸张纹理。默认半径为 2，颜色级数为 8，纸张强度为 0.08，整体强度为 1.0。该方法表达的是屏幕空间保边平滑和纸张质感，不模拟真实颜料在纤维中的流动和边缘积色。

5.8 内部纹理 PNG 导出

导出器只接受 PF_R8G8B8A8_UNORM、完整 mip 0、单数组层的非空二维纹理。程序根据纹理字节数分配 CPU 数组，记录纹理复制命令，提交后同步队列，再在 CPU 上处理 RGBA 数据。红、绿、蓝通道按 sRGB 标准分段函数转换 [12]:

$$C_{\text{sRGB}} = \begin{cases} 12.92C_{\text{linear}}, & C_{\text{linear}} \leq 0.0031308, \\ 1.055C_{\text{linear}}^{1/2.4} - 0.055, & C_{\text{linear}} > 0.0031308. \end{cases} \quad (5.12)$$

Alpha 通道保持不变。PNG 使用 `stb_image_write` 编码 [13]。程序先写入目标路径附加 `.tmp` 的临时文件，编码成功后再替换正式目标，并重新打开结果检查文件大小是否大于零。这样可以避免编码失败时留下看似存在但内容不完整的目标文件。

导出请求可由编辑器 Scene Export 面板或命令行触发。命令行参数包括输出路径、最小化、预热帧数和保存后退出。正式结果使用 Toon 着色和描边，并选择一种后处理。以下命令以像素化为例：

Listing 1 最小化内部纹理导出命令示例

```
1 MoerEditor.exe \
2   --raster-export <output.png> \
3   --raster-export-minimized \
4   --raster-export-warmup 1200 \
5   --raster-export-exit \
6   --raster-shading toon \
7   --raster-stylized outline,pixel \
8   --raster-outline-source both
```

窗口最小化时程序不从单帧渲染函数提前返回，而是在帧末尾根据 `skip_present` 决定是否调用交换链 `Present`。交换链和呈现语义遵循 Vulkan 窗口系统集成机制 [14]。

6 实验过程

6.1 场景准备

运行配置使用 Vulkan 光栅化后端，窗口尺寸和内部输出分辨率均为 1920×1080，默认场景指向 `sponza_mizuki.gltf`。组合场景保留 Sponza 的建筑结构和材质，同时把预变换后的 Mizuki 作为同一网格的附加图元，使引擎启动时一次性加载完整场景。相机位于中央通道正前方，角色正面朝向相机，构图在所有对比实验中保持不变。

正式结果均通过内部纹理导出获得。程序以 Debug 配置运行，导出分辨率为 1920×1080。

6.2 参数设置

表 6.1 汇总本报告使用的核心参数。同类结果只改变待验证模块，其他场景条件保持一致。

表 6.1 主要实验参数

模块	参数	取值
Toon	明暗阈值 / 暗部比例	0.45 / 0.45
描边	宽度 / 深度阈值 / 法线阈值	1 px / 0.02 / 0.15
像素化	像素块尺寸	4 px
半调	强度 / 单元尺寸 / 角度 / 柔化	1.0 / 5 px / 45° / 0.75
故障艺术	强度 / 色差 / 条带尺寸	0.8 / 3 px / 24 px
素描	强度 / 排线间距 / 边缘强度	1.0 / 11 px / 1.5
水彩	强度 / 半径 / 色阶 / 纸张强度	1.0 / 2 / 8 / 0.08
导出	分辨率 / 预热帧数	1920×1080 / 1,200

6.3 实验步骤

实验按照以下顺序执行：

1. 构建 MoerEngine Debug 目标，确认 HLSL 能够由 DXC 编译为 SPIR-V。
2. 检查运行配置中的 Vulkan 后端、1920×1080 分辨率和组合场景路径。
3. 以最小化导出参数启动程序，分别设置 PBR、Toon 和 Toon + 深度/法线描边模式，作为逐层消融对照。
4. 使用统一的 Toon 与描边参数，分别执行像素化、半调、故障艺术、素描和水彩。
5. 每次等待窗口确认最小化并预热 1,200 帧，再从内部最终纹理导出 1920×1080 PNG。

7 实验结果与分析

7.1 PBR、Toon 与描边对比

图 7.1 使用同一组合场景比较三种基础结果。PBR 图保留连续光照、镜面反射和场景中的彩色间接照明，角色面部和建筑表面具有显著亮度渐变。Toon 图将方向光响应压缩为两个明暗区间，墙面和帷幕的连续明暗被简化，角色整体更接近平涂色块。加入深度与法线描边后，拱门、柱体、帷幕、吊灯和角色外轮廓出现黑色边界；法线差还补充了建筑内部折角和角色服装分区。



图 7.1 相同 Sponza + Mizuki 场景的基础着色对比

PBR图显示彩色局部照明、镜面高光和连续材质反射。Toon 图主要使用方向光辐射，画面由稳定的明暗色块表达。描边图的室内结构更清晰，但植物等高频几何也会产生较密集黑线，体现了屏幕空间阈值对复杂场景的敏感性。PBR 和纯 Toon 用于分阶段对照；后续五种结果使用相同的 Toon 与描边参数。

7.2 五种单项后处理结果

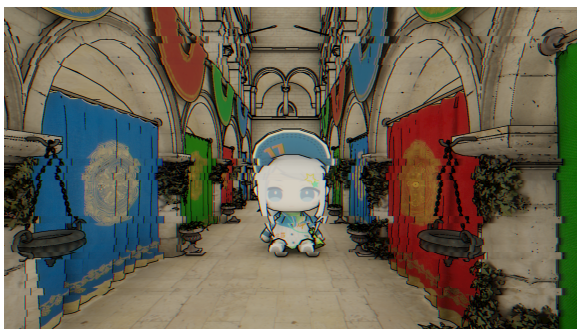
图 7.2 展示五种扩展效果。所有图均使用相同相机、Sponza + Mizuki 场景以及 Toon 与描边参数，每幅图只改变一种后处理。



(a) Toon +描边+像素化



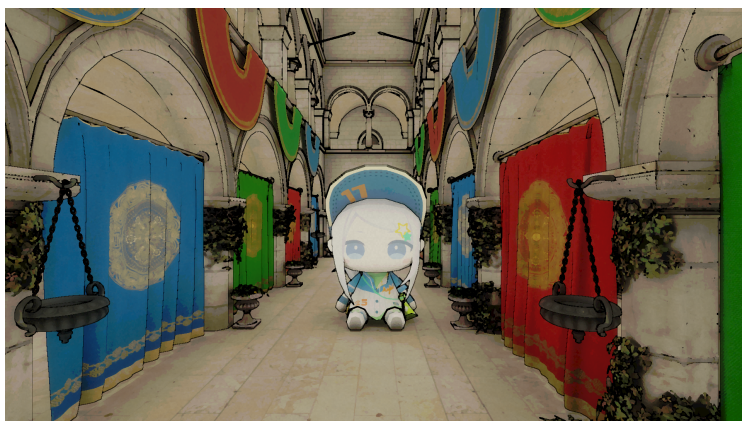
(b) Toon +描边+半调



(c) Toon +描边+故障艺术



(d) Toon +描边+素描



(e) Toon +描边+水彩

图 7.2 Toon 与描边基础上的五种单项后处理结果

五张图的输入均已包含 Toon 平坦明暗区和深度/法线轮廓，随后由单项后处理改变整幅颜色结果。像素化把帷幕纹章、石墙纹理、角色五官和已有轮廓共同离散为规则方块；默认块大小为 4 px，因此整体构图仍可辨识。故障艺术对颜色和轮廓同步施加水平条带位移与 RGB 色散，使数字信号异常效果覆盖完整的必做渲染结果。

素描将输入画面转为浅色纸张、灰度底稿和交叉排线，角色与建筑边界参与铅笔效果生成。半调以纸张底色和网点重构输入，较暗的区域形成更多或更大的网点。水彩执行保边平滑、色阶量化和纸张噪声，使场景颜色与轮廓共同呈现水彩化特征。五种结果均建立在相同的 Toon 与描边基线上。

8 问题分析与解决方法

8.1 Toon 输出接近黑色

Toon 路径需要与引擎的高动态范围 (High Dynamic Range, HDR) 光照量级匹配。仅输出基础色乘二分色阶会使结果明显偏暗。当前实现遍历方向光, 将光源强度与颜色按 $1/\pi$ 缩放, 再与基础色和色阶相乘, 使 Toon 结果在既有曝光链路中保持正常亮度。

8.2 最小化后只显示天空

最小化导出的早期结果只剩天空。检查后发现, 相机宽高比使用编辑器 Scene Color 面板尺寸; 窗口最小化后面板尺寸缩小, 使相机投影和视锥剔除条件改变, 场景物体被错误排除。处理方式是在导出激活期间, 将 Scene Color 分辨率固定为完整渲染分辨率, 再执行相机更新。修正后, 最小化导出的 Sponza 建筑和 Mizuki 角色均完整出现。

9 实验局限与改进方向

实验局限如下:

1. 屏幕空间描边依赖输出分辨率、深度阈值和法线阈值。植物和细密装饰可能产生过多轮廓, 远距离小几何也可能因采样半径不足而丢失边缘。后续可根据深度和视距自适应调整阈值与宽度。
2. 半调在网点重构过程中会合并部分局部细节。后续可增加分色印刷式多角度网屏或基于局部对比度的单元尺寸调节。
3. 素描排线使用固定屏幕方向, 不随物体曲率、主方向或纹理流向变化。后续可根据法线变化或曲率场生成方向纹理。
4. 水彩采用屏幕空间桑原滤镜, 只能实现保边平滑、色阶和纸张噪声, 不能模拟真实颜料扩散、边缘积色和纸张纤维中的流体行为。
5. PNG 导出仅支持 PF_R8G8B8A8_UNORM 的完整二维 mip 0。同步回读会阻塞导出帧, 尚未实现异步批量导出队列和 HDR 文件格式。

10 实验结论

本实验在 MoerEngine Vulkan 光栅化路径中完成了二次元角色非真实感渲染。Mizuki 角色能够在 Sponza 组合场景中正确显示; 延迟光照实现二分色阶 Toon 着色, 屏幕空间处理使用线性深度差和法线夹角差完成角色与场景描边。像素化、半调、故障艺术、素描和水彩五种效果均以独立模块实现。

半调画面由纸张底色和网点重构, 网点数量与面积反映局部明暗。五种正式结果使用相同的 Toon 与描边基线, 分别展示像素化、半调、故障艺术、素描和水彩。复杂

Sponza 背景证明这些效果不仅作用于角色，也能处理建筑、帷幕、植物和吊灯等多尺度内容。

图形输出链路包含内部纹理 PNG 导出分支。导出器在编辑器界面合成之前读取最终场景纹理，完成 GPU 回读、线性颜色到 sRGB 转换、PNG 临时文件编码和完整性检查。本实验完成了课程作业的必做内容以及五种风格化后处理。

11 源码仓库

实验完整实现已公开于 <https://github.com/FeNgkAIY/MoerEngine-NPR>，对应版本为 npr-final-project 分支的 5de587c5 提交。

12 课程总结与建议

本课程围绕物理渲染建立了从几何变换、相机成像、光线追踪和材料外观，到蒙特卡洛积分与渲染方程的完整知识体系。通过本次大作业，对实时光栅化流程、Toon 着色、屏幕空间处理，以及物理渲染与非真实感渲染之间的区别有了更具体的理解，并将课程中的基础概念应用到了实际引擎功能的实现中。

建议课程在现有个人作业之外增加组队任务，由多名同学共同完成规模更大的渲染项目。组队形式可以覆盖场景系统、材料与光照、渲染管线、性能优化和工具界面等不同模块，使课程实践更接近完整图形项目的协作与实现过程。

资源来源与许可

- MoerEngine: 南京大学计算机图形学相关团队维护的实时渲染引擎，源代码采用 Apache-2.0 License[1]。
- Mizuki: 模型 “With This Pitch...! Mizuki” 来自 [Sketchfab 模型页面](#)，作者为雨宮 レン（[作者主页](#)）[2]。
- Sponza: 实验背景使用 MoerEngine 本地资源中的 Sponza 场景。
- stb: PNG 编码使用 stb_image_write，项目允许选择 MIT License 或 Public Domain[13]。
- 课程参考教程：像素化、故障艺术、半调和二次元 NPR 资料分别见参考文献 [5, 7, 9, 10]。

参考文献

- [1] NJUCG. Moerengine: A real-time rendering engine. <https://github.com/NJUCG/MoerEngine>, n.d. 实验基线提交 e89d299f, 访问日期: 2026-07-15.
- [2] 雨宮レン. With this pitch...! mizuki. Sketchfab, <https://sketchfab.com/3d-models/with-this-pitch-mizuki-ab09c162b37c45fdbdb350ac4206b107>, n.d. 访问日期: 2026-07-15.
- [3] Amy Gooch, Bruce Gooch, Peter Shirley, and Elaine Cohen. A non-photorealistic lighting model for automatic technical illustration. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, pages 447–452, 1998. doi: 10.1145/280814.280950.
- [4] Philippe Decaudin. Cartoon-looking rendering of 3d-scenes. Technical Report 2919, INRIA, 1996. URL <https://inria.hal.science/inria-00073720>.
- [5] 燕生. 二次元艺术的渲染技术与风格化突破. 知乎专栏, <https://zhuanlan.zhihu.com/p/1959583904580216424>, n.d. 访问日期: 2026-07-14.
- [6] Takafumi Saito and Tokiichiro Takahashi. Comprehensible rendering of 3-d shapes. In *Proceedings of SIGGRAPH 1990*, volume 24, pages 197–206, 1990. doi: 10.1145/97880.97901.
- [7] 哔哩哔哩视频作者. Unity 教程: 实现 3d 模型像素化——《a short hike》风格模仿. 哔哩哔哩, BV1MQ4y117e4, <https://www.bilibili.com/video/BV1MQ4y117e4>, n.d.. 访问日期: 2026-07-14.
- [8] Bryce E. Bayer. An optimum method for two-level rendition of continuous-tone pictures. In *IEEE International Conference on Communications*, pages 26–11–26–15, 1973.
- [9] Aya Magician. 从仿色图案 (dither pattern) 到半调 (halftone). 知乎专栏, <https://zhuanlan.zhihu.com/p/60325181>, 2019. 访问日期: 2026-07-14.
- [10] 哔哩哔哩视频作者. 简单到爆炸的故障风效果教程. 哔哩哔哩, BV1244y1M7Md, <https://www.bilibili.com/video/BV1244y1M7Md>, n.d.. 访问日期: 2026-07-14.
- [11] Michiyoshi Kuwahara, K. Hachimura, S. Eiho, and M. Kinoshita. Processing of ri-angiocardigraphic images. In *Digital Processing of Biomedical Images*, pages 187–202. Plenum Press, New York, 1976.

- [12] International Electrotechnical Commission. Iec 61966-2-1:1999 multimedia systems and equipment—colour measurement and management—part 2-1: Default rgb colour space—srgb. International Standard, 1999.
- [13] Sean Barrett and contributors. stb single-file public domain libraries for c/c++. <https://github.com/nothings/stb>, n.d. MIT License 或 Public Domain, 访问日期: 2026-07-15.
- [14] Khronos Group. Vulkan specification. <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html>, 2026. 访问日期: 2026-07-15.